

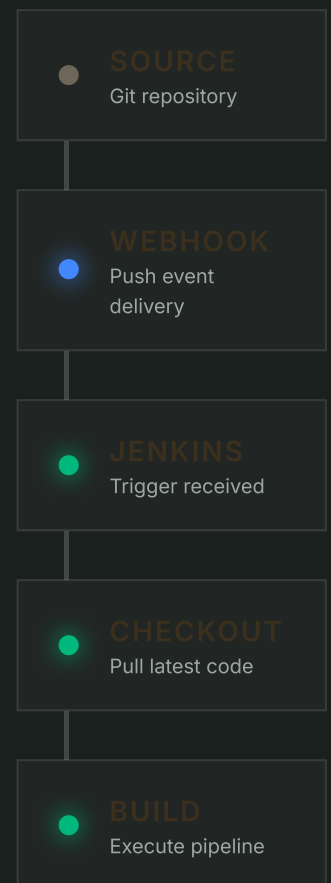
[Up next in Hands-on Labs →](#)**Lab 02: Container Delivery**[Hands-on lab](#) · [45 min](#) · [Env: See book overview](#) · [Overview & prereqs](#) · [Download PDF](#) ·[Reset checks](#)

CI/CD ENGINEERING

From Commit to Production:
Automated Triggering & Webhooks

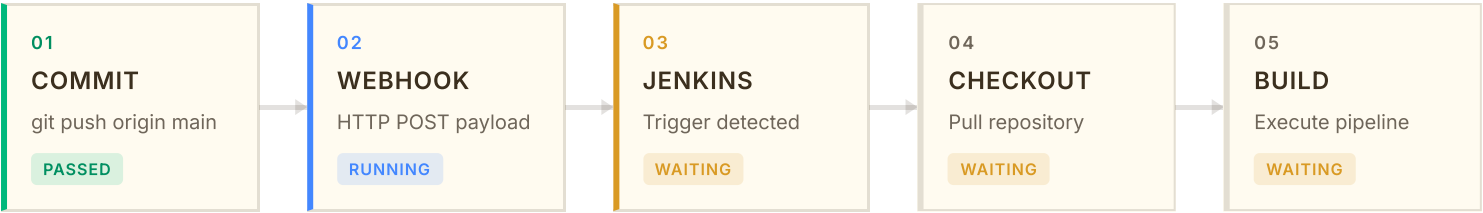
CONTINUOUS INTEGRATION
CONTINUOUS DELIVERY
CONTINUOUS DEPLOYMENT

Nabawy



01 / Automated Integration Foundations

A pipeline is not magic. It is a deterministic sequence of machines making decisions about your code. However, for this sequence to be truly continuous, the transition between a developer's commit and the pipeline execution must be automatic. This is achieved through webhooks.



WHAT

A webhook is a custom HTTP callback triggered by an event, such as a git push. When the event occurs, the source system (GitHub) sends an HTTP POST request to the target system (Jenkins).

WHY

Polling (Jenkins constantly asking GitHub "any changes?") is inefficient and introduces latency. Webhooks provide instant, event-driven execution with zero wasted resources.

WHEN

Whenever a change is pushed to the repository. This includes commits to main, feature branches, or tagged releases, depending on the configured filter.

WHAT CHANGES

The development loop tightens. Feedback on a broken commit returns in minutes, not hours. The integration of code becomes a continuous, automatic flow rather than a manual chore.

WHY THIS EXISTS

Without automated triggering, a developer must manually notify the CI system that new code exists. This context switch breaks flow and inevitably leads to forgotten builds and integration delays. The webhook closes the gap between intent and execution.

02 / Pre-Flight Analysis

Before configuring a single setting, verify that all tools, credentials, and network paths are operational. A failure in webhook delivery is silent—Jenkins simply never runs. Resolve all dependencies in this phase before executing the setup.

ENVIRONMENT VERIFICATION

```
$ curl -s -o /dev/null -w "%{http_code}" http://localhost:8080
200
$ gh auth status
Logged in to github.com as dev-user
$ ngrok http 8080
Session Status online
Forwarding http://1a2b-3c4d.ngrok.io → http://localhost:8080
```

DEPENDENCY CHECK

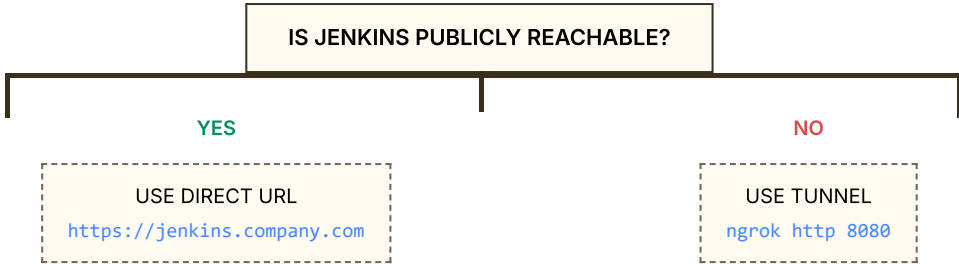
Jenkins Admin Access	CONFIRMED	Request access from Jenkins admin	
REQUIREMENT	STATUS	ACTION IF MISSING	
Public Jenkins URL	CONFIRMED	Start ngrok or configure reverse proxy	
Target Repository URL	CONFIRMED	Identify repo and default branch (main/master)	

GOTCHA

If Jenkins is running locally (WSL, Docker, or localhost), GitHub cannot reach it. A tunneling tool like ngrok is mandatory to expose Jenkins to the public internet. Do not proceed to webhook configuration without a public URL.

03 / Reachability & Network Paths

The webhook requires a clear HTTP path from GitHub's servers to your Jenkins instance. This is the most common point of failure in CI/CD automation. Determine your network topology before setting the Payload URL.



ESTABLISHING THE TUNNEL

If Jenkins is behind a NAT or local firewall, route traffic through a tunnel. This provides a temporary public URL that forwards incoming HTTP requests directly to your local port.

COMMAND

```
$ ngrok http 8080
```

RESULT

```
Forwarding
https://1a2b-3c4d.ngrok.io
→ http://localhost:8080
```

INTERVIEW CHECK

Q: Why can't I just use my local IP for the webhook?

A: GitHub's webhooks are sent from their public cloud infrastructure. Your local machine's IP (e.g., 192.168.x.x) is not routable from the public internet. A tunnel or public IP is required to bridge the connection.

04 / Job & Credential Setup

Build the Jenkins pipeline structure and configure authenticated access to the GitHub repository. Never use raw GitHub passwords; always use a Personal Access Token (PAT) stored securely in Jenkins.

STEP 1: CREATE PIPELINE JOB

```
Jenkins Dashboard → New Item
> Name: github-webhook-pipeline
> Type: Pipeline
Pipeline Definition: Pipeline script
Initial Script:
pipeline {
  agent any
  stages { stage('Test') { steps { echo 'Build OK' } } }
}
```

STEP 2: CONFIGURE SECURE CREDENTIALS

Navigate to **Pipeline Syntax** → **checkout: Check out from version control**. Generate the Groovy snippet using a new credential of type **Username with password**. Use your GitHub username as the username, and the PAT as the password.

GOTCHA

GitHub has deprecated password authentication for Git operations. Using your account password will result in a 403 Forbidden error during checkout. The PAT acts as a scoped, revocable proxy for your password.

INTERVIEW CHECK

Q: What is the difference between a PAT and a password?

A: A password grants full access to an account. A PAT is a scoped token that limits access to specific repositories and actions, can have an expiration date, and can be revoked without changing the account password.

05 / Lab: Secure Checkout

Execute the authenticated checkout configuration. This lab verifies that Jenkins can securely access the private repository using the generated PAT credentials.

- OBSERVE

The pipeline needs to pull source code from a private repository. Anonymous access is disabled. We need to verify the credential binding works.
- EXECUTE

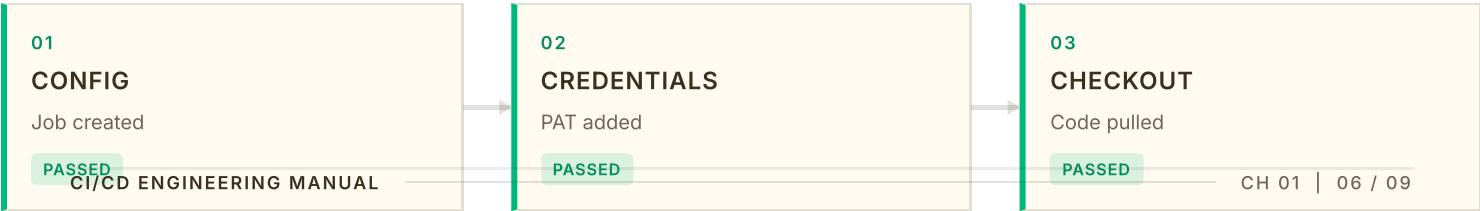
In Jenkins Job Configuration, paste the generated Groovy checkout snippet into the Pipeline script block. Ensure the `credentialsId` matches the ID of the created PAT credential (e.g., `github-token`).
- VERIFY

Save the job and click "Build Now". Check the Console Output for the git fetch commands succeeding without authentication errors.
- EXPLAIN

Jenkins injected the PAT into the Git command environment, allowing the fetch to succeed over HTTPS. The workspace now contains the repository files.

GENERATED GROOVY SNIPPET

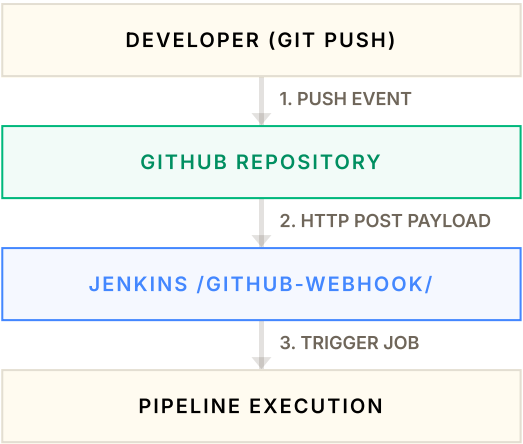
```
checkout([
    $class: 'GitSCM',
    branches: [[name: '*/main']],
    userRemoteConfigs: [[
        url: 'https://github.com/user/repo.git',
        credentialsId: 'github-token'
    ]]
])
```



06 / The Webhook Bridge

With the pipeline ready, establish the automated trigger. This requires configuring the webhook on GitHub and enabling the corresponding listener on Jenkins.

ARCHITECTURE FLOW



STEP 5: GITHUB CONFIGURATION

In GitHub, navigate to **Repo Settings** → **Webhooks** → **Add webhook**. Set the Payload URL to the Jenkins endpoint. The trailing slash is critical.

```
Payload URL:
https://github-webhook/
Content type: application/json
Trigger: Just the push event
```

GOTCHA

The URL `/github-webhook/` must include the trailing slash. Without it, Jenkins' routing will fail and return a 404, causing GitHub to mark the webhook as failed (red X).

STEP 6: JENKINS TRIGGER CONFIGURATION

In the Jenkins job configuration, scroll to **Build Triggers**. Enable **GitHub hook trigger for GITScm polling**. This tells Jenkins to react when the webhook payload is received, rather than polling on a schedule.

07 / Lab: End-to-End Trigger

Test the complete automation loop. A commit pushed to GitHub should automatically trigger the Jenkins pipeline without any manual intervention.

- OBSERVE

The webhook is configured, and the Jenkins trigger is enabled. We need to verify the event-driven connection.
- EXECUTE

Make a trivial change to the repository (e.g., add a comment to a file). Commit and push to the main branch.
- VERIFY

Watch the Jenkins dashboard. A new build should appear in the Build History within seconds, marked as triggered by "GitHub push".
- PROOF

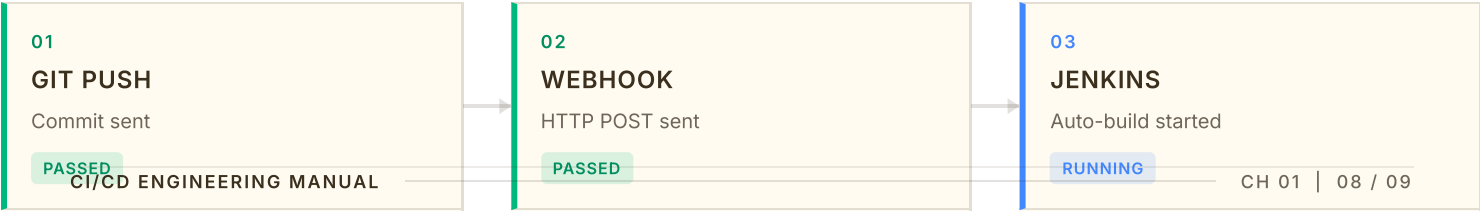
The automation loop is closed. Human intervention is no longer required for integration. The pipeline is truly continuous.

COMMAND

```
$ echo "// update" >> index.html
$ git add .
$ git commit -m "test trigger"
$ git push origin main
```

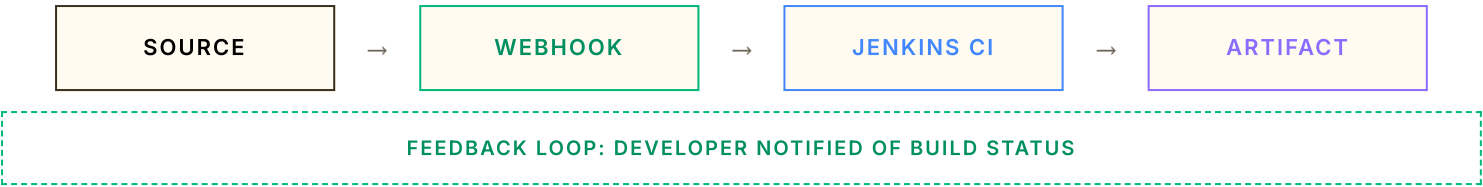
RESULT

```
Enumerating objects: 5, done.
Writing objects: 100% (3/3), 340 bytes.
To github.com:user/repo.git
e7f8a9c..d1c2b3a main -> main
```



08 / System Map & Troubleshooting

The automated trigger is a critical link in the broader deployment lifecycle. If it fails, the entire pipeline stalls. Use this map to contextualize the webhook and troubleshoot failures.



TROUBLESHOOTING FLOW

Jenkins never triggers	Webhook not delivered	Check GitHub Webhook "Recent Deliveries" for HTTP errors	
SYMPTOM	ROOT CAUSE	RECOVERY	
Checkout fails (403)	Invalid PAT or expired	Regenerate PAT in GitHub, update Jenkins Credentials	
Checkout fails (branch)	Wrong branch name	Verify default branch (main vs master)	

DEV	Automated CI Trigger	PASSED
STAGING	Auto-Deploy on Success	RUNNING
PROD	Manual Approval Gate	BLOCKED

CI/CD ENGINEERING MANUALCH 01 | 09 / 09

Up next in Hands-on Labs →

Lab 02: Container Delivery

RELATED BOOKS

Lab 02: Container Delivery
50 min · INT
→ Next in path

Lab 08: Backup & Restore
60 min · ADV
→ Previous in path

Jenkins Labs
180 min · ADV
→ Shared tags

Lab 03: Multibranch Foundations
45 min · INT
→ Shared tags